



Article

On the Proof of Ownership of Digital Wallets

Chen Wang¹ , Zi-Yuan Liu¹ and Masahiro Mambo^{2,*}

¹ Graduate School of Natural Science and Technology, Kanazawa University, Kanazawa 920-1192, Japan; wang880093@stu.kanazawa-u.ac.jp (C.W.); ziyaunliu@stu.kanazawa-u.ac.jp (Z.-Y.L.)

² Institute of Science and Engineering, Kanazawa University, Kanazawa 920-1192, Japan

* Correspondence: mambo@ec.t.kanazawa-u.ac.jp

Abstract: With the widespread adoption and increasing application of blockchain technology, cryptocurrency wallets used in Bitcoin and Ethereum play a crucial role in facilitating decentralized asset management and secure transactions. However, wallet security relies heavily on private keys, with insufficient attention to the risks of theft and exposure. To address this issue, Chaum et al. (ACNS'21) proposed a “proof of ownership” method using a “backup key” to prove ownership of private keys even when exposed. However, their interactive proof approach is inefficient in large-scale systems and vulnerable to side-channel attacks due to the long key generation time. Other related schemes also suffer from low efficiency and complex key management, increasing the difficulty of securely storing backup keys. In this paper, we present an efficient, non-interactive proof generation approach for ownership of secret keys using a single backup key. Our approach leverages non-interactive zero-knowledge proofs and symmetric encryption, allowing users to generate multiple proofs with one fixed backup key, simplifying key management. Additionally, our scheme resists quantum attacks and provides a fallback signature. Our new scheme can be proved to capture unforgeability under the computational indistinguishability from the Uniformly Random Distribution property of a proper hash function and soundness in the quantum random oracle model. Experimental results indicate that our approach achieves a short key generation time and enables an efficient proof generation scheme in large-scale decentralized systems. Compared with state-of-the-art schemes, our approach is applicable to a broader range of scenarios due to its non-interactive nature, short key generation time, high efficiency, and simplified key management system.



on Elliptic Curve Cryptography (ECC) [5]. These are collectively referred to as classical digital signature algorithms. With the development of large, fault-tolerant quantum computers, such classical digital signature schemes face significant threats. Shor's algorithm [6] can solve the discrete logarithm and integer factorization problems in $O(\log N)$ polynomial time N . This implies that any adversary possessing a large enough quantum computer can compute a user's private key within a feasible time once the public key is given. As a result, the adversary can generate valid signatures of the victim user. Recent studies, such as those in [7–9], have explored applying post-quantum cryptographic schemes. Although such schemes enable users to protect their digital assets against quantum attacks, users may fail or simply forget to migrate their classical digital signature-protected assets to systems secured by post-quantum cryptographic schemes. By the time they realize the issue, their secret key might already have been compromised. In such cases, post-quantum signatures and their implementations are not sufficient to address the problem.

In the scenario described above, proving the ownership of digital wallets becomes crucial for users. To prove ownership, simply employing zero-knowledge proofs, such as zero-knowledge proof of knowledge (ZKPK) [10] protocols, is not suitable because an adversary with knowledge of the stolen secret key can also generate a valid ZKPK proof. To address this issue, Chaum et al. (ACNS'21) [11] introduced the concepts of "proof of ownership" and "fallback signature," allowing users to prove ownership of secret keys and generate fallback signatures even when their secret keys are publicly exposed in a decentralized environment. To realize these concepts, they proposed the scheme S_{leeve} , a combination of the ECDSA signature with a hash-based eW-OTS⁺ digital signature (an extension of the W-OTS⁺ signature proposed by Chaum). The security definition of a proof of ownership framework is that the probability of an adversary without knowledge of the secret witness successfully forging a valid proof is negligible, which means that the proof scheme is sound. This implies that adversaries cannot forge a valid proof even if they successfully break the classical digital signature, such as ECDSA. In the S_{leeve} scheme, the proof system enables a user to generate a proof using a nested eW-OTS⁺ digital signature scheme, which signs an unpredictable challenge generated by the verifier. Accordingly, if this post-quantum hash-based signature is unforgeable, it is infeasible for adversaries to forge the signature, thereby ensuring the soundness of the proof scheme. In their paper, the unforgeability is proved under the assumption of the second-preimage resistance of a hash function. However, their proof does not consider the quantum random oracle model (QROM). Unlike them, we directly prove the soundness of our proof scheme under QROM, allowing adversaries to perform quantum queries on random oracles.

offline environment, our scheme provides easier and more efficient storage and retrieval compared with existing schemes.

Regarding the first issue we mentioned above, a natural idea to improve the S_{leeve} scheme is to transmit the involved interactive proof scheme into a non-interactive one employing the Fiat–Shamir transformation [15]. However, such a transformation does not address the key generation time and backup key storage issues. Considering all these three issues, following the concept of proof of ownership, we propose an approach utilizing zero-knowledge proof and symmetric encryption techniques. Specifically, our construction generates a non-interactive, post-quantum zero-knowledge proof of the relationship between the nested backup key and the secret key as the proof of ownership, enabling the users to non-interactively generate proofs of ownership of the secret keys without requiring inputs from other verifiers. In such non-interactive schemes, an adversary observing a valid proof could replicate it. Then, with cloned proof and public parameters, the adversary could pretend to other verifiers that he generated the valid proof. This potential clone attack could be addressed using Certification Authority (CA) systems and additional specific authentication algorithms. As this paper focuses solely on the proof of ownership scheme, concerns related to clone attacks are only mentioned here and not further explored.

Furthermore, our scheme requires only one fixed backup key to generate proofs of ownership for multiple secret keys, enabling a simplified key management system, especially when large-scale proofs of ownership for the secret keys are needed. Our protocol can also be nested within classical digital signature schemes in the real world and offers a more efficient key generation process than the S_{leeve} scheme. Our contributions are as follows.

- We propose a new efficient non-interactive proof of ownership scheme, featuring a simplified key management system while supporting a post-quantum fallback digital signature mechanism.
- We prove that the signing scheme in our approach achieves unforgeability under the computational indistinguishability from the Uniformly Random Distribution property of a proper hash function and that the proof scheme is sound in the quantum random oracle model.
- With evaluation, compared with state-of-the-art schemes, our scheme has a relatively efficient key generation phase and the most efficient proof generation phase in a large-scale decentralized system.

2. Background

2.1. Digital Signature

We describe a simple scenario involving two users, Alice and Bob, where Alice wants to send a message m to Bob securely. Alice wants to ensure that the message received by Bob is unchanged and that Bob can verify that the message was sent by Alice. Bob wants

The correctness requires that, for any security parameter n and any message m ,

$$\Pr[\text{Verify}(m, \text{pk}, \sigma) = 0 \mid (\text{sk}, \text{pk}) \leftarrow \text{Gen}(1^n), \sigma \leftarrow \text{Sign}(\text{sk}, m)] \leq \text{negl}(n) \quad (1)$$

For a classical digital signature, like ECDSA, unforgeability is a crucial security property, typically demonstrated through the EUF-CMA security game. We present the definition of EUF-CMA 1.

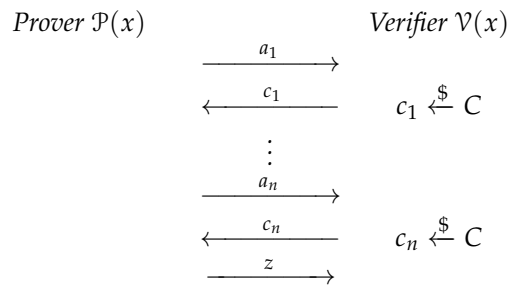
Definition 2 (EUF-CMA1 Security). *Given a signature scheme $\text{sig} = (\text{Gen}, \text{Sign}, \text{Verify})$ and the security parameter n , we say that sig is EUF-CMA1-secure if any PPT algorithm $\mathcal{A}$ has negligible advantage in the EUF-CMA1 game, defined as*

$$\text{Adv}_{\mathcal{A}}^{\text{EUF-CMA1}} = \Pr[\text{Verify}(\text{pk}, m^*, \sigma^*) = 1 \wedge m^* \notin Q : (\text{sk}, \text{pk}) \leftarrow \text{Gen}(1^n), (m^*, \sigma^*) \leftarrow \mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}(\text{pk})] \quad (2)$$

where $\mathcal{A}^{\text{Sign}(\text{sk}, \cdot)}$ denotes the adversary's access to the signing oracle with private key sk for each fresh message, and Q denotes the set of messages m queried by $\mathcal{A}$.

To prove that our proof of ownership signature scheme achieves unforgeability, we assume an extra property for the hash function in our scheme.

Definition 3 (Indistinguishability). *For security parameter $n</$*



Verifier $\mathcal{V}$ accept if $V(x, a_1, \dots, a_n, c_n, z) = 1$ is satisfied

where $a_1 \dots a_n$ are commitments generated by the prover and $c_1 \dots c_n$ are challenges randomly sampled from C by the verifier, z is the final output from the prover, and predicate $V(x, a_1, \dots, a_n, c_n, z)$ outputs 1 only if the proof $(x, a_1, \dots, a_n, c_n, z)$ is valid.

Then, the definition of a general Fiat–Shamir transformation for a PCIP system (including the ZKIP system) is demonstrated as

Definition 5 (Fiat–Shamir transformation for general PCIP (mFS)). *Given an $(2n + 1)$ -move PCIP $\Pi = (\mathcal{P}, \mathcal{V})$, a hash function H with appropriate domain and range in C , and a verification predicate V_{FS} , we define the non-interactive proof system $FS[\Pi] = (\mathcal{P}_{FS}^H, \mathcal{V}_{FS}^H)$ as follows. The prover $\mathcal{P}_{FS}^H$ outputs*

$$(x, a_1, \dots, a_n, z)$$

Based on MPC-in-the-Head (MPCitH) techniques, the Banquet protocol constructs a 7-move interactive identification scheme for proving knowledge. This scheme can be converted into a non-interactive proof using the Fiat–Shamir transform, a method believed to be secure against quantum attacks. The simplified scheme operates as follows: given a key pair consisting of a random secret key $sk = k$ and a public key $pk = (x, y)$ such that $AES_k(x) = y$, signing a message m involves a proof generation algorithm *Prove*, generating a non-interactive zero-knowledge proof of knowledge of k in a way that binds m to the proof. The verifier’s challenges are derived from random oracles, with the message and public key as initial input. This ensures that these parameters are tied to the signature. Given an AES function $AES_k(x): \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$, which presents as a binary circuit, the Banquet scheme is described by three algorithms:

- $Gen_b(1^n)$: given security parameter n as input, sample $x \in \mathcal{X} = \{0, 1\}^n$ and $k \in \mathcal{K} = \{0, 1\}^n$, and compute $y = AES_k(x)$, outputs public key $pk \leftarrow (x, y)$ and secret key $sk \leftarrow k$.
- $Sign_b(sk, m)$: given a message m and secret key sk as input and compute $\sigma \leftarrow \text{Prove}(sk, m)$, output σ .
- $Verify_b(pk, m, \sigma)$: given signature σ and public key pk as input, compute the verification algorithm $Verify(pk, \sigma, m)$, which outputs 1 if the result of *Verify* is 1; otherwise, output 0.

2.3. Proof of Ownership of Digital Signature Scheme

In classical digital signature schemes, security relies on the secrecy of the private key. An adversary can forge signatures on arbitrary messages if the secret key is revealed. As discussed in Section 1, an attacker can apply a quantum attack to compromise the secret key of the classical asymmetric signature scheme. In this scenario, the classical digital signature is no

for all probabilities are computed over the random coins of the generation and proof verification algorithm and adversary. c^* is the challenge generated by the adversary to generate π^* .

3. Related Work

In this paper, we address the scenario of a stolen secret key from classical digital signatures and propose a method to generate a post-quantum proof. The instinctive approach to proving the ownership of a secret in the digital signature is to use a replacement or additional secure algorithm. At this point of writing, many researchers have proposed a “drop-in replacement” in the form of a software library or hardware module. But since these algorithms have unique resources, performance, and platform considerations [30,31], coupled with insufficient proof of knowledge [32], they still need more extensions to be designed and carried out. Although fail-stop signature schemes [9] provide a forgery-proof mechanism, they neither prove ownership of the secret key nor provide fallback mechanisms, highlighting the need for additional improvements.

Another approach is to use a backup key that overrides the regular signing key when the secret key is leaked. In Chaum et al. [11], the researchers propose the use of a hash-based W-OTS⁺ scheme to generate a proof for the regular secret key, which can be used as a backup signature scheme when the regular key is compromised or lost. However, the S_{leeve} scheme limits the scenario of longer-term non-interactive protection of data because the proof generation of a S_{leeve} scheme requires a challenge as a common input to compute the proof, which can also be treated as an interactive identification scheme. Moreover, the key generation time of the S_{leeve} scheme is 5–10 times slower than ECDSA, and it is hard to apply the S_{leeve} scheme as a classical digital signature scheme due to the low efficiency.

In [33], researchers enhance the security of digital signature schemes by randomly choosing a pre-image value related to the secret key in the key generation step and generating two zero-knowledge proofs of the public key generation. Although the authors aim to create a post-quantum extension scheme for the classical digital signature schemes, their scheme can also be utilized to prove the ownership of a revealed secret key. Compared with our scheme, the authors attempt to compute the secret key of an ECDSA encryption scheme with a corresponding pre-image value utilizing the proper hash function and generating two zero-knowledge proofs for the hash computation and elliptic curve computation. However, the extended signature scheme needs to be simplified. We argue that the zero-knowledge proof for the public

- ECDSA: ECDSA is commonly implemented in digital wallets as a signing scheme due to its use of an unpredictable random number generator for private key generation. This generator produces outputs that perfectly match the output space of a one-way hash function, which is also widely used in digital wallets.
- Hash function: In our approach, the hash function's output space is a finite field that aligns well with the curve in ECDSA.
- AES function: AES is a 128-bit block cipher based on a substitution–permutation network. The key length is allowed to be 128, 192, or 256 bits and can be considered as a 4×4 matrix of elements in $\mathbb{F}_{2^8}$. The AES S-box computes a multiplicative inverse in the field $\mathbb{F}_{2^8}$. In our case, the key k of the AES function should be properly selected so the key generation repeats the AES computation $y = \text{AES}_k(x)$ until there are no S-boxes in the computation of y with input 0.
- Zero-knowledge proof: Our approach uses a non-interactive and post-quantum secure zero-knowledge proof generated from the Banquet protocol [22]. We modify Banquet's signing and verification scheme by removing the message input and generating a NIZKPoK for an AES encryption key. In Banquet signature, signing a message m involves adding the message into a hash function of the challenge and verifying it. Without message input, the generated NIZKPoK does not suffer from security or efficiency issues. The zero-knowledge proof demonstrates that for given (x, y) , the prover possesses an AES encryption key k satisfying $\text{AES}_k(x) = y$. The parameters and functions used in the proof are the following: the security parameter n , the total number of parties N in the underlying MPC protocol, the total number of S-boxes L , the number of parallel repetitions τ , the commitment function Commit, challenge computing hash functions (H_1, H_2, H_3) , and the hash expand function Expand and random tape sample function tape.

With the setting definition mentioned above, our protocol can be evaluated as five algorithms: Gen, Sign, Verify, Proof, and Verify–Proof, which will be demonstrated sequentially.

- Realization of Gen: The function Gen, shown in Algorithm 1, is an extended function of the Gen algorithm of the ECDSA. Additional steps are taken to generate a backup key bk and use it to compute the secret key sk and the public key pk . It samples $\text{bk} \leftarrow \{0,$

Algorithm 1 Proof of ownership ECDSA: key generation Gen**Input:** ECC base point G ; ECC order P ; security parameter n **Output:** (bk, sk, pk) Sample a backup key $bk \leftarrow \{0, 1\}^n$ Sample a random value $x \leftarrow \{0, 1\}^n$ Compute the result of AES function $y = \text{AES}_{bk}(x)$ **for** There are zero input S-boxes in AES computation **do** Resample $x, bk \leftarrow \{0, 1\}^n$ Recompute $y = \text{AES}_{bk}(x)$ **end for**Compute secret key $sk = H(x, y)$ Compute public key $pk = sk \cdot G \bmod P$ **return** (bk, sk, pk) **Algorithm 2** Proof of ownership ECDSA: signing Sign**Input:** ECC base point G ; ECC order P ; secret key sk ; message m **Output:** σ

Algorithm 5 Proof of ownership ECDSA: proof verification Verify–Proof**Input:** ECC base point G ; ECC order P ; secret key sk ; public key pk ; proof π **Output:** $result$ Parse $\pi = (\pi_n, x, y)$ Check $Verify(\pi_n) = result$ Compute $sk_v = H(x, y)$ Compute $pk_v = sk_v \cdot G \bmod P$ **if** $result = 1$, $sk_v = sk$, and $pk_v = pk$ **then** **return** 1**else** **return** 0**end if****5. Security of Our Design****5.1. EUF-CMA1 Security of Signing Scheme**

Firstly, similar to the proof in [35], we aim to prove that our signature scheme, whose main signature algorithms are Algorithms 1–3, is EUF-CMA1 secure. The security of the ECDSA scheme is given by [36]. Although many signature schemes are proven unforgeable under the random oracle model, our proof framework requires only the

- Procedure Init: the Init procedure takes a security parameter n as input, generates (bk, sk, pk) as Algorithm 1 with security parameter n , generates an empty list $\mathcal{L}$ to store relevant information, and publishes the public key pk as output.
- Procedure Sign: A forger chooses a message m and sends it to the Sign procedure as input. Sign checks if the message m is already in the list $\mathcal{L}$. If the message is in the list, Sign will abort. Otherwise, Sign will sign the message using the secret key sk generated in Init, add the message m to the list $\mathcal{L}$, and send the signature σ back to the forger as output.
- Procedure Fin: The forger attempts to generate a valid message and signature pair (m^*, σ^*) and sends it to the procedure Fin as input. Fin first checks if the message m^* has already been on the list $\mathcal{L}$. If the message is in the list, Fin will abort. Otherwise, Fin verifies the message and signature pair by checking if $\text{Verify}(pk, m^*, \sigma^*) = 1$. If verification fails, Fin will abort the procedure. If verification is successful, Fin returns 1 as a result.

Proof. Assume the existence of a forger $\mathcal{F}$ of our signature scheme, which wins the game from Algorithm 6 by outputting a forgery (m^*, σ^*) with non-negligible probability. Then, we construct a PPT distinguisher algorithm D , which breaks the indistinguishability property of H with high probability. We construct D as follows:

- D receives the string x of Definition 3 as input. The string x is generated in one of two ways: either $x \leftarrow H(M)$, where H is a hash function applied to the arbitrarily chosen message M , or by $x \leftarrow \mathcal{U}$, where $\mathcal{U}$ represents a uniform distribution.
- In Algorithm 1, users compute the secret key using a hash function. With the input string x , D can modify the key generation algorithm by setting the secret key sk' as the string x and computing the public key as $pk' = sk' \cdot G \bmod P$.
- Considering that the backup key bk is

Then, the probability definition of indistinguishability $|\Pr[x \leftarrow H(M), D(x) = 1] - \Pr[x \leftarrow \mathcal{U}, D(x) = 1]|$ with the distinguisher D we constructed can be transformed as

$$\begin{aligned} & |\Pr[x \leftarrow H(M), D(x) = 1] - \Pr[x \leftarrow \mathcal{U}, D(x) = 1]| \\ &= |\Pr[x \leftarrow H(M), \text{Verify}(m^*, \sigma^*, \text{pk}') = 1] - \Pr[x \leftarrow \mathcal{U}, \text{Verify}(m^*, \sigma^*, \text{pk}') = 1]| \\ &= |\Pr[x \leftarrow H(M), \text{Verify}(m^*, \sigma^*, \text{pk}') = 1] - \text{negl}(n)|. \end{aligned}$$

If there exists a forger $\mathcal{F}$ of our signature scheme, it wins the EUF-CMA1 game from Algorithm 6 by outputting a forgery (m^*, σ^*) with non-negligible probability. Then, we have that $\Pr[x \leftarrow H(M), \text{Verify}(m^*, \sigma^*, \text{pk}') = 1]$ is non-negligible. Therefore, $|\Pr[x \leftarrow H(M), \text{Verify}(m^*, \sigma^*, \text{pk}') = 1] - \text{negl}(n)|$ is non-negligible, and $|\Pr[x \leftarrow H(M), D(x) = 1] - \Pr[x \leftarrow \mathcal{U}, D(x) = 1]|$ is non-negligible, which is contrary to the assumption that the hash function in our scheme holds indistinguishability. $\square$

5.2. Security of Proof Scheme

In the proof generation function, recall that the input is (bk, x) from the key generation phase. Compute $\text{AES}_{\text{bk}}(x) = y$, then generate the NIZKPoK proof π_n and output $\pi = (x, y, \pi_n)$ (Algorithms 1, 4 and 5). The soundness of the NIZKPoK proof system relies on the Fiat–Shamir transform [20,37] and obtains a better concrete-security reduction than [38].

is a one-way function, then the AES function used in our scheme is also a one-way function, and for all polynomial time adversaries A :

$$\Pr[x \leftarrow \{0,1\}^n, k \leftarrow \mathcal{K}', y = \text{AES}_k(x), k' = A(x, y) : y = \text{AES}_{k'}(x)] \leq \text{negl}(n). \quad (6)$$

5.3. Strong Unforgeability

In our design, we have analyzed the signing scheme using the EUF-CMA1 definition and, therefore, do not rule out the case that an attacker can find a new signature for a previously signed message. For instance, an attacker may manipulate a signature generated by the signer and, using some method, pass the verification with the same message. For strong unforgeability, we modify our signing scheme by combining Algorithms 2 and 4, as well as Algorithms 3 and 5. Then, we have that even if the ECDSA signature generated by Algorithm 3 is manipulated and passes the pre-verification of the ECDSA scheme in Algorithm 5, an attacker can not pass the verification because the verification of NIZKPoK is required. Regarding the NIZKPoK scheme, similar to the Banquet scheme, it requires the following:

- It is difficult to find a set of intermediate seeds in the tree that derive the same leaf seeds used in a signature, which means that more security concerns should be needed in seed sampling.
- It is difficult to find a different seed that produces the same outputs when input to the Expand function utilized as a seed generation function. In our scheme, Expand is instantiated by hash functions, and we assume that the hash function is a random oracle to meet the requirement, like the assumption in Definition 3.

With the above assumptions, our scheme is supposed to have strong unforgeability.

5.4. Post-Quantum Security

Our scheme

Remark 1. x° holds that $(x^\circ, w^\circ) \in R$ for the relation R of our NIZKPoK and underlying ZKIP. S^A simulates a prover in the underlying ZKIP system, and $\mathcal{V}$ is a verifier outputting the verification result v . In our NIZKPoK, V_{FS}^H is the verification predicate, and π is composed of $(a_1, c_1, \dots, a_n, c_n, z)$ according to Definition 5.

Proof. Compared with the Banquet, our NIZKPoK removes the message m as hash input during the computation of the first challenge. Accordingly, the Fiat–Shamir transformation of our NIZKPoK, as defined in Definition 5, aligns with the Fiat–Shamir transformation for general PCIP (mFS). To generate the reduction in mFS, the challenges of Fiat–Shamir transformation are required to use distinct inputs to H with probability 1 for the application of the simulator from [21], Theorem 7. In our case, this distinctness is ensured by the input of commitment. Similarly to the construction in [22], the commitments of our NIZKPoK are generated separately and have different sizes, ensuring distinct inputs for different challenge computations. $\square$

Using the reduction from Lemma 2, we can prove that our NIZKPoK system holds soundness in QROM with the following lemma.

Lemma 3. Assume that the underlying ZKIP system has (statistical or computational) soundness. Then, in QROM, our NIZKPoK system also has (statistical or computational) soundness.

Proof. From Lemma 2, given that q is polynomial in the security parameter, the success probability of any dishonest prover $A_{NIZKPoK}$ against our NIZKPoK and the success probability of a dishonest prover A_{ZKIP} challenging the underlying ZKIP are polynomially related. If our NIZKPoK does not have soundness in QROM, the success probability of $A_{NIZKPoK}$ is non-negligible. Then, the polynomially related success probability of A_{ZKIP} is also non-negligible, which is contrary to the assumption that the

wallet user to sign secret values even under a secret key revealing attack. Compared with the Banquet scheme, our scheme modifies the key generation phase with the input (x, k) generated from Algorithm 1, which satisfies the security requirement. We briefly describe it with three algorithms: Gen, Sign, Verify:

- Gen_f : given (bk, x, y) from Algorithm 1 and as input, set secret key $sk \leftarrow bk$, public key $pk \leftarrow (x, y)$, output sk, pk .
- $\text{Sign}_f(sk, m)$: given a message m and secret key sk as input, compute $\sigma \leftarrow \text{Prove}(sk, m)$, output σ .
- $\text{Verify}_f(pk, m, \sigma)$: given signature σ and public key pk as input, compute a verification algorithm $\text{Verify}(pk, \sigma, m)$, that outputs 1 if the result of Verify is 1; otherwise, output 0.

The security of our fallback scheme derives from the Banquet signature scheme of Baum et al. [22]. They have proven that assuming that the functions used in the Signature Generation phase Commit, H_1 , H_2 , and H_3 are modeled as random oracles, Expand is a pseudorandom generator (PRG) with output computationally ϵ_{PRG} -close to uniform; the seed tree construction is computationally hiding, the execution and security parameters are appropriately chosen, and the key generation function $f_x : sk \mapsto pk$ is a one-way function; the Banquet signature is EUF-CMA secure. Sign_f and Verify_f in our fallback scheme are the same as Sign_b and Verify_b , and Gen_f receives bk, x, y from Algorithm 1, satisfying that $f_x : bk \mapsto (x, y)$ is a one-way function. Then, our fallback scheme is also EUF-CMA secure, like the Banquet signature. To prove the EUF-CMA security of our fallback scheme in QROM, the most promising approach seems to be the general result for multi-round Fiat–Shamir type signatures

ECDSA key generation algorithm, which is significantly slower than our scheme and the scheme of Tan and Zhou.

Table 1. Key generation phase.

Methods	Operations	Time (ms)
Our Scheme	$exp + 3AES + hash$	1.09
Our Scheme (with fixed bk)	$exp + AES + hash$	0.97
Tan and Zhou scheme	$exp + hash$	0.91
S_{leeve}	$exp + 1329hash$	4.48
ECDSA	exp	0.84

Let exp represent a Elliptic Curve Scalar Multiplication (ECSM) operation of secp256r1, let AES represent a AES function operation of AES-128, and $hash$ represent a hash function operation of SHA-256.

We also evaluate the proof scheme of our scheme, S_{leeve} , and the Tan and Zhou scheme [33] with a theoretical analysis in Table 2. Our analysis reveals that our scheme significantly reduces the proof size by half and approximately doubles the computational efficiency compared with the scheme presented in [33]. Given that t_H , t_E , and t_A constitute only a minor portion of the overall computational costs, the computational cost and the proof size of all schemes are primarily determined by the methods employed in each scheme. Thus, the comparative analysis between our scheme and S_{leeve} can be effectively reduced to a comparison of their respective methodologies. In our scheme, the proof generation and verification times are similar to the signing and verification times of the Banquet signature scheme. According to [22], Banquet has a signing time of 6.36 ms and a verification time of 4.68 ms for the L1 security level. In contrast, the eWOTS⁺ signature scheme, analyzed in [11], has signing and verification times of 0.024 ms and 1.472 ms, respectively. Given that eWOTS⁺ is

key. This can solve security problems such as hard forks and revocation of wallet addresses. Compared with the proof of ownership scheme S_{leeve} , our design provides non-trivial long-term data protection and data migration capabilities. This ensures enhanced security and resilience in maintaining the integrity and ownership of digital assets over time. Furthermore, our scheme prevents backup key leakage when a user needs to prove the ownership of multiple secret keys.

Digital wallets: If there is evidence of a quantum attack against a digital wallet environment, such as a blockchain system, users can utilize our construction to prove the ownership of potentially compromised tokens by generating and publishing a NIZKPoK for the secret key without any challenge from other verifiers and redeem tokens in an alternative manner while the token is not locked up. Our scheme can also be utilized when a digital wallet user suffers a quantum attack and needs to revoke the ownership of an address the adversary has stolen. Suppose a user's ECDSA key is stolen; in that case, they can revoke the wallet address by creating proof of ownership with a backup key, informing the network, and non-interactively being verified by other users. The user can then introduce a new wallet address generated with the same backup key containing the funds associated with the old wallet. We also address scenarios where a user manages multiple secret keys, such as in coin mixing applications [13]. Additional security considerations of the backup key storage from our scheme offer a more secure solution compared with existing state-of-the-art approaches.

Beyond digital wallets: Our scheme can also be applied to sign long-term data requiring non-repudiation protection, such as providing extra security for quantum-resistant, time-stamped signed PDFs in Adobe Acrobat Reader DC. Our scheme can also facilitate efficient encryption scheme migration for classical digital signature schemes. Given that our proof of ownership digital signature scheme and fallback scheme can generate two signatures with the same secret value, input sampling, and message, the migration process only modifies the verification phase. It retains the old encryption without additional inputs. This migration can be carried out in three steps, as defined below:

1. The signing parties sign a document using Algorithm 2, similar to a classical ECDSA scheme, noticing that the document is vulnerable in a post-quantum environment. The verifying parties receive and verify the signed document using Algorithm 3, ensuring that the ECDSA signature scheme correctly signs the document. They then inform the signing parties to initiate the migration process.
2. The signing parties generate a proof of ownership for the secret key using Algorithm 4 and then publish this proof. Verifying parties first ignore the published new signatures and continue verifying old ones. Once the first verification is complete and the proof

for longer-term digital data protection, such as PDF files for important financial contacts and valuable digital arts in blockchain systems. The simplified key management system, short key generation time, and efficient proof scheme make our approach well suited for scenarios in which a user manages multiple secret keys, such as in hierarchical deterministic (HD) wallets.

Our future work is as follows:

- A new zero-knowledge proof scheme: The size and computational time of the proof scheme are linked to the circuit computation in our design. VOLE-in-the-head presents a promising potential solution [42].
- QROM security: In this paper, we proved the soundness of our proof scheme under QROM. However, our work does not provide a concrete proof of the fallback (Banquet signature) scheme under QROM. The QROM security of the Banquet signature could potentially be proven if we can modify the NIZKPoK in the Banquet signature into a Σ -protocol [22]. Another possible method to prove the QROM secure Banquet signature is to follow the proof framework in [43], where they provide a QROM security proof for the Helium proof system [44] with online extractability [45].
- Post-quantum signatures: Our signature is designed to hide any post-quantum signature inside a classical digital signature scheme. So, in addition to NIZK-based signatures, we will implement other types of post-quantum signature schemes, such as lattice-based signatures.

Author Contributions: Conceptualization, C.W., Z.-Y.L. and M.M.; methodology, C.W. and Z.-Y.L.; software, C.W.; validation, C.W., Z.-Y.L. and M.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by JSPS KAKENHI Grant Number JP23K28070 (formerly, JP23H03380).

Data Availability Statement: The data presented in this study are openly available in <https://github.com/wangbabb/poo> (accessed on 13 December 2024).

11. Chaum, D.; Larangeira, M.; Yaksetig, M.; Carter, W. W-OTS+ up my sleeve! A hidden secure fallback for cryptocurrency wallets. In Proceedings of the International Conference on Applied Cryptography and Network Security, Virtual, 21–24 June 2021; Springer: Berlin/Heidelberg, Germany, 2021; pp. 195–219.
12. Standaert, F.X. Introduction to side-channel attacks. In *Secure Integrated Circuits and Systems*; Springer: Boston, MA, USA, 2010; pp. 27–42.
13. Ruffing, T.; Moreno-Sanchez, P.; Kate, A. Coinshuffle: Practical decentralized coin mixing for bitcoin. In Proceedings of the Computer Security-ESORICS 2014: 19th European Symposium on Research in Computer Security, Wroclaw, Poland, 7–11 September 2014; Proceedings, Part II 19; Springer: Berlin/Heidelberg, Germany, 2014; pp. 345–364.
14. Gutoski, G.; Stebila, D. Hierarchical deterministic bitcoin wallets that tolerate key leakage. In Proceedings of the International Conference on Financial Cryptography and Data Security, San Juan, Puerto Rico, 30 January 2015; Springer: Berlin/Heidelberg, Germany, 2015; pp. 497–504.
15. Fiat, A.; Shamir, A. How to prove yourself: Practical solutions to identification and signature problems. In Proceedings of the Conference on the Theory and Application of Cryptographic Techniques, Linz, Austria, 1 January 1986; Springer: Berlin/Heidelberg, Germany, 1986; pp. 186–194.
16. Ishai, Y.; Kushilevitz, E.; Ostrovsky, R.; Sahai, A. Zero-knowledge from secure multiparty computation. In Proceedings of the Thirty-Ninth Annual ACM Symposium on Theory of Computing, San Diego, CA, USA, 11–13 June 2007; pp. 21–30.
17. Giacomelli, I.; Madsen, J.; Orlandi, C. {ZKBoo}: Faster {Zero-Knowledge} for Boolean Circuits. In Proceedings of the 25th USENIX Security Symposium (USENIX Security 16), Austin, TX, USA, 10–12 August 2016; pp. 1069–1083.
18. Chase, M.; Derler, D.; Goldfeder, S.; Katz, J.; Kolesnikov

32. Au, M.H.; Susilo, W.; Mu, Y. Proof-of-knowledge of representation of committed value and its applications. In Proceedings of the Australasian Conference on Information Security and Privacy, Sydney, Australia, 5–7 July 2010; Springer: Berlin/Heidelberg, Germany, 2010; pp. 352–369.
33. Tan, T.G.; Zhou, J. Layering quantum-resistance into classical digital signature algorithms. In Proceedings of the Information Security: 24th International Conference, ISC 2021, Virtual Event, 10–12 November 2021; Proceedings 24; Springer: Berlin/Heidelberg, Germany, 2021; pp. 26–41.
34. Chase, M.; Derler, D.; Goldfeder, S.; Orlandi, C.; Ramacher, S.; Rechberger, C.; Slamanig, D.; Zaverucha, G. Post-quantum zero-knowledge and signatures from symmetric-key primitives. In Proceedings of the 2017 ACM Sigsac Conference on Computer and Communications Security, Dallas, TX, USA, 30 October–3 November 2017; pp. 1825–1842.
35. Chaum, D.; Larangeira, M.; Yaksetig, M. Tweakable Sleeve: A Novel Sleeve Construction Based on Tweakable Hash Functions. In Proceedings of the the International Conference on Mathematical Research for Blockchain Economy, Vilamoura, Portugal, 12–14 July 2022; Springer: Berlin/Heidelberg, Germany, 2022; pp. 169–186.
36. Fersch, M.; Kiltz, E.; Poettering, B. On the one-per-message unforgeability of (EC) DSA and its variants. In Proceedings of the Theory of Cryptography: 15th International Conference, TCC 2017, Baltimore, MD, USA, 12–15 November 2017; Proceedings, Part II 15; Springer: Berlin/Heidelberg, Germany, 2017; pp. 519–534.
37. Pointcheval, D.; Stern, J. Security arguments for digital signatures and blind signatures. *J. Cryptol.* **2000**, *13*, 361–396. [[CrossRef](#)]
38. Katz, J.; Kolesnikov, V.; Wang, X. Improved non-interactive zero knowledge with applications to post-quantum signatures. In Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security, New York, NY, USA, 3 October 2018; pp. 525–537.
- 39